

Pillar 1: Frontend Architecture & System Design

Senior Frontend / Fintech interview answers. Each question includes a practical architecture-focused response with examples and trade-offs where relevant.

1. How do you design a micro-frontend strategy for a banking portal with multiple teams?

I split the portal by business domains such as accounts, payments, loans, risk, reports, and admin. The shell owns authentication, layout, global navigation, observability, feature flags, and route composition. Each team owns its domain micro-frontend end-to-end: UI, tests, release pipeline, monitoring, and documentation. I avoid sharing internal state directly and use contracts such as route parameters, shared events, and typed APIs. For banking, I also enforce security boundaries, consistent design-system usage, independent deployments, and rollback capability.

2. Explain your approach to modernizing a jQuery application into React.

I use the strangler pattern instead of a big-bang rewrite. First I audit the jQuery app, identify risky flows, add tests around current behavior, and isolate areas that can be replaced safely. Then I mount React into small legacy sections and gradually replace jQuery DOM manipulation with state-driven React components. Shared utilities, validation, and API clients are extracted into framework-neutral TypeScript modules. This keeps delivery moving while reducing technical risk.

3. How do you maintain UI consistency across different platforms Web/Mobile?

I maintain consistency through a design system, design tokens, shared interaction patterns, and common accessibility rules. Tokens cover color, typography, spacing, radius, elevation, and motion. Web may consume them as CSS variables or Tailwind tokens; mobile may consume the same values through React Native styles. I do not force identical components across platforms, but I keep the visual language, behavior, wording, and validation rules consistent.

4. Design a component library that supports multiple brand identities.

I separate component behavior from brand styling. Components use semantic tokens such as `--color-primary`, `--surface-card`, and `--text-danger` instead of hardcoded values. Each brand provides a theme file that maps tokens to brand-specific colors, fonts, spacing, and radius. The library includes Storybook documentation, visual regression tests, accessibility checks, and semantic versioning. This allows one Button, Modal, Table, or DatePicker to serve multiple tenants or brands.

5. How do you architect state management for data-heavy financial flows like risk visibility?

I separate server state, client UI state, and global application state. Server state such as portfolios, risk exposure, and transactions is managed by Apollo Client, React Query, or SWR. UI state such as filters, selected rows, and expanded panels stays local, in URL params, or in a small store. Global state contains session, tenant, permissions, theme, and feature flags. This prevents one large store from becoming a bottleneck and makes risk dashboards easier to test and scale.

6. When should a team choose Vue.js over React for enterprise apps?

Vue is a good choice when the team wants a more opinionated, template-driven framework, faster onboarding, and a simpler mental model. React is stronger when the team needs a larger ecosystem, flexible architecture, React Native sharing, or broad hiring availability. For enterprise apps, I would choose based on team skills, ecosystem needs, long-term maintenance, and integration requirements rather than popularity alone.

7. How do you manage shared state across micro-frontends without tight coupling?

I avoid sharing a single global store across all micro-frontends unless absolutely necessary. Shared state should be communicated through stable contracts: URL parameters, custom browser events, backend state, shared auth/session service, or a small typed event bus. Each micro-frontend should own its internal state. For example,

selecting a portfolio can emit a `portfolio:selected` event with a typed payload instead of importing another team's store.

8. What is your strategy for handling legacy technical debt in a fast-paced agile environment?

I classify technical debt by risk: security-critical, business-blocking, and cosmetic. Critical debt gets prioritized immediately; business-blocking debt is attached to feature work; cosmetic debt is handled gradually. I avoid stopping all delivery for a large cleanup. When touching a legacy area, I add tests, improve types, remove duplication, and document decisions. This makes refactoring continuous and business-friendly.

9. How do you design a frontend to be framework-agnostic?

I keep domain logic outside the UI framework. Business rules, validation, API clients, permission checks, formatters, and TypeScript types live in core packages. React, Vue, or mobile apps only consume these packages. For example, `canApproveLoan(userRole, amount)` should be a plain TypeScript function, not a React hook. This makes future migration, testing, and reuse much easier.

10. Explain how you would implement multi-tenancy in a frontend architecture.

I implement tenant resolution early in the bootstrap process. The app identifies the tenant from hostname, path, token claims, or backend configuration. Then it loads tenant-specific theme, feature flags, permissions, locale, API configuration, and route visibility. I keep tenant configuration typed and validated. Sensitive tenant rules should come from the backend, while the frontend uses them to render the correct experience.

11. How do you approach monorepo management for large-scale enterprise projects?

I use a monorepo when multiple apps share UI libraries, API clients, types, lint configs, and utilities. A typical structure has `apps/` for deployable applications and `packages/` for shared libraries. Tools like Nx, Turborepo, pnpm workspaces, or Yarn workspaces help with affected builds, caching, and dependency boundaries. I also enforce ownership, clear import rules, shared CI pipelines, and versioning discipline.

12. What are the trade-offs between Client-Side and Server-Side Rendering for fintech?

CSR is often simpler for authenticated dashboards, real-time tools, and highly interactive internal portals. SSR is useful for public banking pages, SEO, first contentful render, and marketing content. In fintech, SSR becomes complex when pages are highly personalized because caching, authentication, and sensitive data handling must be precise. I usually combine both: SSR for public or semi-static pages and CSR for secure interactive dashboards.

13. How do you ensure WCAG accessibility in complex financial dashboards?

I start with semantic HTML, keyboard navigation, visible focus states, proper labels, color contrast, and meaningful error messages. Complex widgets like grids, modals, menus, and charts need special care. Charts should have text summaries or data tables so users are not forced to rely only on visuals. I also use automated checks such as axe, but still perform manual keyboard and screen-reader testing.

14. How do you design an authentication flow for secure banking applications?

I design authentication as a layered flow: login, MFA or OTP, secure session creation, refresh strategy, idle timeout, absolute timeout, role-based authorization, and audit logging. I avoid storing sensitive tokens in `localStorage` when possible and prefer secure HTTP-only cookies for high-risk apps. The frontend should never be the only security layer; backend authorization must protect every sensitive action.

15. What is your process for choosing a new UI framework such as Tailwind vs Material UI?

I evaluate accessibility, customization, design-system fit, bundle impact, team skills, documentation, community, TypeScript support, and long-term maintenance. Tailwind is excellent for custom brand systems and token-driven UI. Material UI is useful when the team needs many ready-made enterprise components quickly. I usually run a small proof of concept before committing.

16. How do you architect a frontend to support real-time data via WebSockets or GraphQL?

I classify data as real-time, near-real-time, or normal API data. Real-time streams such as market data or risk exposure go through WebSockets or GraphQL subscriptions, then into a normalization layer and cache/store. I include reconnection logic, stale data indicators, fallback polling, throttling, and error handling. The UI should clearly show when data is live, delayed, or disconnected.

17. Describe your experience with technical documentation for architectural decisions.

I use Architecture Decision Records for major decisions. An ADR includes context, decision, alternatives, consequences, owner, and date. This helps teams understand why Apollo, Redux, Module Federation, Tailwind, or a design-system strategy was chosen. Good documentation avoids repeated debates and helps new engineers onboard faster.

18. How do you handle versioning for a shared design system?

I use semantic versioning. Patch versions fix bugs, minor versions add backward-compatible features, and major versions introduce breaking changes. I also maintain a changelog, migration guide, deprecation policy, Storybook examples, and release notes. Breaking changes should be planned and communicated early because many apps may depend on the design system.

19. How do you manage cross-cutting concerns like logging and error tracking?

I centralize cross-cutting concerns through shared services and providers. API errors go through interceptors, UI crashes go through Error Boundaries, and production issues go to tools such as Sentry, Datadog, or New Relic. Logs must include useful metadata such as route, release version, and browser, but must never expose sensitive banking data.

20. Explain the role of a BFF in modern architecture.

A Backend for Frontend sits between frontend and backend services. It shapes APIs according to frontend needs, combines multiple service calls, reduces over-fetching, hides backend complexity, and can enforce security rules. For example, a banking dashboard may need accounts, cards, user permissions, and notifications. The BFF can return a single optimized response for the frontend.

21. How do you ensure modularization in a massive codebase to prevent spaghetti code?

I structure code by domain features and enforce import boundaries. Feature modules own their routes, components, hooks, API calls, and tests. Shared code is promoted only when it is truly reusable. I avoid deep imports across features and use lint rules or workspace tools to enforce boundaries. This keeps the codebase understandable as teams grow.

22. What are the challenges of SSR in a highly personalized banking environment?

SSR is challenging because every user may have different permissions, account data, menu items, and dashboard content. This reduces cacheability and increases risk of data leakage if caching is wrong. Hydration mismatch can also happen when server and client state differ. For banking, SSR must carefully handle authentication, authorization, headers, cookies, and sensitive data.

23. How do you design for offline-first capabilities in mobile banking?

I support offline only for safe use cases such as viewing cached transactions, drafting forms, reading help content, or preparing but not submitting requests. Sensitive actions such as money transfer or approval submission should require online validation. Offline storage must be encrypted, synced carefully, and cleared on logout. Conflict resolution and stale-data warnings are essential.

24. How do you balance innovation with the stability required for finance platforms?

I separate experimentation from critical flows. Stable flows such as login, payments, approvals, and card replacement need strict testing, monitoring, and rollback plans. Innovation like AI assistants, new dashboards, or personalization can be released behind feature flags and canary deployments. This lets the team move forward

without risking the core banking experience.

25. Describe your role in mentoring junior devs on architectural best practices.

I mentor by explaining reasoning, not only giving instructions. I guide juniors on component boundaries, naming, test strategy, accessibility, state management, and trade-offs. In code reviews, I explain why a change improves maintainability or reduces risk. I also encourage small design documents before large changes so juniors learn to think architecturally.